

Chapter 3

Diffusion Models

Exam Study Guide, Worked Concepts, Sample Questions, and

Prepared from	uploaded course materials
Main source	Chapter_3_Diffusion_Models(1).ipynb
Exam source	Chapter_3_Summary_and_Sample_Questions.pdf
Practice source	Chapter_3_Flashcards.pdf

How to use it

Study diffusion as a two-process model: a fixed analytic forward noising process and a learned reverse denoising process. Memorize the difference between a full trajectory and the closed-form marginal $q(x_t|x_0)$.

Study order: First read the concept sections, then solve the five sample exam questions, then test yourself with the flashcards.

Contents

- 1. Quick exam map
- 2. Core idea of diffusion models
- 3. Forward Markov noising process
- 4. Trajectory distribution versus marginal $q(x_t|x_0)$
- 5. Why the reverse process must be learned
- 6. Noise prediction and timestep conditioning
- 7. U-Nets, schedules, DDPM, and DDIM
- 8. Formula cheat sheet and symbol key
- 9. Homework and code prompts
- 10. Exam-style questions
- 11. Flashcards

Most useful mental shortcut

Diffusion generation is easier because the model learns many small denoising steps from Gaussian noise to data instead of one giant jump.

1. Quick exam map

Diffusion models generate by reversing a gradual corruption process. They first define how data becomes noise, then train a neural network to reverse that path.

Clean data x_0

real image or data sample

Forward q

add Gaussian noise step by step

Noisy x_t

partially corrupted sample at time t Denoiser ϵ_θ predict added noise or denoising direction Reverse p_θ iteratively sample back to clean data Figure: the main method pipeline for this chapter.

Exam topic	What you must be able to say
Forward process	A fixed Markov chain gradually corrupts x_0 into noise.
Reverse process	A learned model approximates $p_\theta(x_{t-1} x_t)$.
Trajectory vs. marginal $q(x_{1:T} x_0)$ is the full path; $q(x_t x_0)$ is one noisy state.	Noise schedule
β_t controls how much information is removed at each step.	Network target
Common DDPM training predicts the added noise ϵ .	Time conditioning

The denoiser must know how noisy x_t is.

2. Zero-prior explanation: understand the

idea before the equations This section assumes you have not read the lecture yet. The goal is to build intuition first, then connect the intuition to the equations and exam language.

Core idea: generation by reversing corruption Rather than map random noise to a realistic sample in one step, diffusion breaks the job into many small denoising steps. This makes learning more manageable.

Forward Markov process

The forward process q is chosen by the designer. Each step adds a small amount of Gaussian noise, and after many steps the data becomes approximately standard Gaussian.

Trajectory versus marginal

The full trajectory $q(x_{1:T}|x_0)$ includes every noisy state. Training usually needs only $q(x_t|x_0)$, which can be sampled directly without simulating every previous step.

Memory line
Diffusion is controlled destruction plus learned reconstruction.

3. Core concepts and worked explanations

Why reverse is learned

The forward rule is known, but the exact reverse conditional depends on the unknown data distribution. A neural network learns an approximation to that reverse direction.

Noise prediction and time conditioning

DDPM commonly trains $\epsilon_\theta(x_t, t)$ to predict the noise ϵ . The timestep t is crucial because denoising a nearly clean image is different from denoising almost pure noise.

DDPM versus DDIM

DDPM sampling is stochastic; DDIM can follow a more deterministic path and often uses fewer sampling steps. Training and sampling choices are related but not identical.

4. Method explanation, worked example,

and application

Step	Method explanation
Choose schedule	Define β_t , α_t , and cumulative $\bar{\alpha}_t$.
Sample training pair	Pick x_0 , t , and ϵ , then construct x_t directly.
Predict noise	Train $\epsilon_\theta(x_t, t)$ to estimate ϵ .

Step	Method explanation
Generate iteratively	Start from noise and run reverse steps to x_0 .

Worked example

Example: at a medium timestep, an image is partly corrupted. The network learns what noise component should be removed. At small t it restores details; at large t it must infer global structure from near-noise.

Applications and usage

Usage: diffusion is used in image generation, editing, inpainting, super-resolution, audio generation, video generation, and scientific inverse problems.

5. Formula / definition cheat sheet

Read formulas as compact sentences. Do not memorize symbols blindly; use the symbol key table after the formula table to understand what each part represents.

Formula	Interpretation
$q(x_t x_{t-1}) = N(\sqrt{\alpha_t} x_{t-1}, \beta_t I)$	One forward noising step.
$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$	Cumulative signal strength.
$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1-\bar{\alpha}_t)} \epsilon$	Direct noisy sample.
	$L = E[\epsilon - \epsilon_\theta(x_t, t) ^2]$ Noise-prediction training loss.

6. Equation symbol key table

This table explains the main symbols used in the equations. When studying, cover the meaning column and check whether you can explain each symbol clearly.

Symbol	Meaning	Plain-English reading
x_0	Clean data	Original image/sample before noising.
x_t	Noisy sample	Data after t noising steps.
t	Timestep	Noise level index.
β_t	Noise schedule value	Amount of noise added at step t .
α_t	Signal retention	One minus β_t .
$\bar{\alpha}_t$	Cumulative signal retention	How much original signal remains after
t steps.	ϵ	Gaussian noise
Random noise used to corrupt x_0 .	ϵ_θ	Neural prediction

Model estimate of the added noise.

7. Main code prompt / implementation

skeleton This is not meant to replace the lecture code. It is a compact prompt showing the computational pattern you should recognize in homework and exams.

DDPM training prompt

```
x0 = batch noise = randn_like(x0) t = randint(1, T) xt = sqrt(alpha_bar[t]) * x0 + sqrt(1-alpha_bar[t]) * noise
noise_pred = model(xt, t)
loss = mse(noise_pred, noise)
```

How to explain the code in words

Start by identifying the input, the model output, the objective being minimized, and the update rule. Then connect the code back to the formulas and the method table.

8. Common exam traps

Trap	Correct way to think
Thinking the forward process is	learned Forward noising is fixed analytically; reverse denoising is learned.
Confusing trajectory and	marginal The full path and one noisy state are different objects.
Ignoring timestep t	The denoising task depends strongly on the noise level.

9. Exam-style sample questions with

answers These are written in the style of conceptual exam answers. Use them to practice explaining, not only recognizing terms.

Question 1

What is the difference between $q(x_{1:T}|x_0)$ and $q(x_t|x_0)$?

Sample answer

The first is the full noising path; the second is one noisy state at a chosen timestep.

Question 2

Why is the reverse conditional not directly usable?

Sample answer

It depends on the unknown real data distribution, so a neural network approximates it.

Question 3

Why does predicting ϵ make sense?

Sample answer

Under Gaussian corruption, ϵ identifies the noise component and gives denoising direction.

Question 4

Why must the denoiser know t ?

Sample answer

Different timesteps require different denoising behavior.

Question 5

How do DDPM and DDIM differ conceptually?

Sample answer

DDPM uses stochastic reverse transitions; DDIM can use more deterministic, faster trajectories.

10. Flashcards for rapid review

Use these for active recall. Cover the answer column and try to answer before checking.

Card 1

Q: What is diffusion generation?

A: Iterative denoising from noise to data.

Card 2

Q: Is the forward process learned?

A: No, it is fixed analytically.

Card 3

Q: What does β_t control?

A: Noise added at each step.

Card 4

Q: What does ϵ_θ predict?

A: The added noise.

Card 5

Q: Why condition on t ?

A: Noise level changes the task.

Card 6

Q: What is $q(x_t|x_0)$?

A: Closed-form noisy marginal.

Card 7

Q: What is DDIM useful for?

A: Faster deterministic-style sampling.

Card 8

Q: Why use a U-Net?

A: It captures local detail and global context.

Card 9

Q: What must you remember about Forward process?

A: A fixed Markov chain gradually corrupts x_0 into noise.

Card 10

Q: What must you remember about Reverse process?

A: A learned model approximates $p_\theta(x_{t-1}|x_t)$.

Card 11

Q: What must you remember about Trajectory vs. marginal?

A: $q(x_{1:T}|x_0)$ is the full path; $q(x_t|x_0)$ is one noisy state.

Card 12

Q: What must you remember about Noise schedule?

A: β_t controls how much information is removed at each step.

Card 13

Q: What must you remember about Network target?

A: Common DDPM training predicts the added noise ϵ .

Card 14

Q: What must you remember about Time conditioning?

A: The denoiser must know how noisy x_t is.

Card 15

Q: What does this formula represent:

$q(x_t|x_{t-1}) = N(\sqrt{\alpha_t} x_{t-1}, \beta_t)$?

A: One forward noising step.

Card 16

Q: What does this formula represent: $\alpha_t = 1 - \beta_t$?

A: Signal kept at step t .

Card 17

Q: What does this formula represent:

$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$?

A: Cumulative signal strength.

Card 18

Q: What does this formula represent:

$q(x_t|x_0) = N(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)\mathbb{I})$?

A: Closed-form marginal.

Card 19

Q: What does this formula represent: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$?

A: Direct noisy sample.

Card 20

Q: What does this formula represent:

$L = E[||\epsilon - \epsilon_\theta(x_t, t)||^2]$?

A: Noise-prediction training loss.

Card 21

Q: What does x_0 mean?

A: Clean data. Original image/sample before noising.

Card 22

Q: What does x_t mean?

A: Noisy sample. Data after t noising steps.

Card 23

Q: What does t mean?

A: Timestep. Noise level index.

Card 24

Q: What does β_t mean?

A: Noise schedule value. Amount of noise added at step t .

Card 25

Q: What does α_t mean?

A: Signal retention. One minus β_t .

Card 26

Q: What does $\bar{\alpha}_t$ mean?

A: Cumulative signal retention. How much original signal remains after t steps.

Card 27

Q: What does ϵ mean?

A: Gaussian noise. Random noise used to corrupt x_0 .

Card 28

Q: What does ϵ_θ mean?

A: Neural prediction. Model estimate of the added noise.

Card 29

Q: What is the key idea of Core idea: generation by reversing corruption?

A: Rather than map random noise to a realistic sample in one step, diffusion breaks the job into many small denoising steps. This makes learning more

Card 30

Q: What is the key idea of Forward Markov process?

A: The forward process q is chosen by the designer. Each step adds a small amount of Gaussian noise, and after many steps the data becomes approximately standard

Card 31

Q: What is the key idea of Trajectory versus marginal?

A: The full trajectory $q(x_{1:T}|x_0)$ includes every noisy state. Training usually needs only $q(x_t|x_0)$, which can be sampled directly without simulating every previous

Card 32

Q: What is the key idea of Why reverse is learned?

A: The forward rule is known, but the exact reverse conditional depends on the unknown data distribution. A neural network learns an approximation to that reverse

Card 33

Q: What is the key idea of Noise prediction and time conditioning?

A: DDPM commonly trains $\epsilon_\theta(x_t, t)$ to predict the noise ϵ . The timestep t is crucial because denoising a nearly clean image is different from denoising almost pure

Card 34

Q: What is the key idea of DDPM versus DDIM?

A: DDPM sampling is stochastic; DDIM can follow a more deterministic path and often uses fewer sampling steps. Training and sampling choices are related but not

Card 35

Q: What happens in the step: Choose schedule?

A: Define β_t , α_t , and cumulative $\bar{\alpha}_t$.

Card 36

Q: What happens in the step: Sample training pair?

A: Pick x_0 , t , and ϵ , then construct x_t directly.

Card 37

Q: What happens in the step: Predict noise?

A: Train $\epsilon_\theta(x_t, t)$ to estimate ϵ .

Card 38

Q: What happens in the step: Generate iteratively?

A: Start from noise and run reverse steps to x_0 .

Card 39

Q: Can you explain forward vs reverse?

A: Forward corrupts; reverse is learned denoising.

Card 40

Q: Can you write x_t from x_0 directly?

A: Use the closed-form marginal formula.

Card 41

Q: Can you explain noise prediction?

A: Model estimates ϵ used to corrupt x_0 .

Card 42

Q: Can you explain why sampling is slow?

A: Many reverse steps may be required.

Card 43

Q: What is the difference between $q(x_{1:T}|x_0)$ and $q(x_t|x_0)$?

A: The first is the full noising path; the second is one noisy state at a chosen timestep.

Card 44

Q: Why is the reverse conditional not directly usable?

A: It depends on the unknown real data distribution, so a neural network approximates it.

Card 45

Q: Why does predicting ϵ make sense?

A: Under Gaussian corruption, ϵ identifies the noise component and gives denoising direction.

Card 46

Q: Why must the denoiser know t ?

A: Different timesteps require different denoising behavior.

Card 47

Q: How do DDPM and DDIM differ conceptually?

A: DDPM uses stochastic reverse transitions; DDIM can use more deterministic, faster trajectories.

Card 48

Q: In plain English, why is $q(x_t|x_{t-1}) = N(\sqrt{\alpha_t} x_{t-1}, \beta_t I)$ important?

A: It captures: One forward noising step. Use it as a compact exam sentence, then define each symbol.

Card 49

Q: In plain English, why is $\alpha_t = 1 - \beta_t$ important?

A: It captures: Signal kept at step t . Use it as a compact exam sentence, then define each symbol.

Card 50

Q: In plain English, why is $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ important?

A: It captures: Cumulative signal strength. Use it as a compact exam sentence, then define each symbol.

Card 51

Q: In plain English, why is $q(x_t|x_0) = N(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$ important?

A: It captures: Closed-form marginal. Use it as a compact exam sentence, then define each symbol.

Card 52

Q: In plain English, why is $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ important?

A: It captures: Direct noisy sample. Use it as a compact exam sentence, then define each symbol.

Card 53

Q: In plain English, why is $L = E[||\epsilon - \epsilon\theta(x_t, t)||^2]$ important?

A: It captures: Noise-prediction training loss. Use it as a compact exam sentence, then define each symbol.

Card 54

Q: In plain English, why is $q(x_t|x_{t-1}) = N(\sqrt{\alpha_t} x_{t-1}, \beta_t I)$ important?

A: It captures: One forward noising step. Use it as a compact exam sentence, then define each symbol.

Card 55

Q: In plain English, why is $\alpha_t = 1 - \beta_t$ important?

A: It captures: Signal kept at step t . Use it as a compact exam sentence, then define each symbol.

Explanation

Chapter 3: Diffusion Models

How this section is organized

Each item below is a heading taken from the corresponding Jupyter notebook in the same order it appears there. The explanation beside it tells you what that notebook section is trying to teach and why it matters for the review.

Source notebook

Chapter_3_Diffusion_Models(1).ipynb

Headings included

28

Notebook headings with explanation

1. Chapter 3 - Diffusion Models

This chapter is a gentle introduction to diffusion models (mostly DDMP) By the end of this chapter, you should be able to: 1. Understand the basic idea behind Diffusion generative processes 3.

2. Review: What Is a Markov Chain?

A Markov chain is a sequence of random variables $x_0, x_1, x_2, \dots$ that satisfies the Markov property: $p(x_t \mid x_{t-1}, x_{t-2}, \dots, x_0) = p(x_t \mid x_{t-1})$. In words: > The future depends only on the present, not on the past.

3. Basic Idea Behind Diffusion Models

The goal of a diffusion model is to start from a simple distribution (typically standard Gaussian noise) and generate samples that look like data from a target distribution. In other words, we want a procedure that takes $x_T \sim N(0, I)$ and produces a final sample x_0 whose distribution matches the data $x_0 \sim q_{\text{data}}(x)$.

4. Diffusion models: the main families

The term "diffusion model" now covers several closely-related families. We will primarily focus on DDPM-style Gaussian diffusion (discrete-time), but the broader landscape is given in table below.

5. A table of common diffusion families

Family / Name; Forward corruption; What is learned; Sampling style; Typical use / notes DDPM (Denoising Diffusion Probabilistic Models); Gaussian Markov chain (VP); $\epsilon_{\theta}(x_t, t)$ or $\mu_{\theta}(x_t, t)$; Stochastic, many steps; Canonical diffusion; ELBO motivates loss; strong quality DDIM (Implicit diffusion); Same as DDPM; Same network as DDPM; Deterministic or low-noise, fewer steps; Sampling method (often 10-100 steps) without retraining Score-SDE (continuous-time score-based); SDE (VP / VE /).

6. Notation

It's easy to get lost in notation like $p_{\theta}(x_{0:T})$ or $q(x_{1:T} \mid x_0)$. This notation is extremely important to understand correctly.

7. Summary

The trajectory describes randomness over time. - The marginal describes uncertainty at a fixed time. - Diffusion training only needs the marginal. - Sampling uses the full reverse trajectory.

8. Forward and Reverse Transitions - Core Markov Structure

At the heart of diffusion models is a Markov chain. Forward (Noising) Transition The forward diffusion process is defined step-by-step as: $q(x_t \mid x_{t-1}) = N(\alpha_t x_{t-1}, (1-\alpha_t)\sigma_t^2)$ Equivalently, sampling can be written as: $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t} \sigma_t \epsilon$, $\epsilon \sim N(0, I)$.

9. Simple Example of the forward diffusion process

This heading introduces a practical example or code activity. Its purpose is to connect the theory to an executable workflow or visualization. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

10. Data to be sampled from	We create a 1D mixture distribution as the target data set. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
11. Defining the forward diffusion process for the 1D data	This notebook heading introduces defining the forward diffusion process for the 1d data and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
12. We can visualize how the forward process morphs the original distribution into $N(0,1)$	This notebook heading introduces we can visualize how the forward process morphs the original distribution into $n(0,1)$ and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
13. Mutual information decay: why x_t “forgets” x_0	A precise way to say “the forward process destroys information” is to look at the mutual information: $I(X_0; X_t)$. - If $I(X_0; X_t)$ is large, then knowing x_t tells you a lot about x_0 . - If $I(X_0; X_t)$ is near 0, then x_t carries almost no information about x_0 . In forward diffusion, x_t becomes increasingly noisy, so $I(X_0; X_t)$ should decrease with t . The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
14. Time scheduling in diffusion: Comparing linear vs cosine	The noise schedule (often specified via β_t) controls how quickly the forward process destroys information. A convenient way to compare schedules is through: - The cumulative product $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$; $\alpha_t = 1 - \beta_t$ - The signal-to-noise ratio (SNR) at time t : $\text{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
15. Reversing the Markov chain in Diffusion	Diffusion can be understood as a two-part story: We create a forward Markov chain by noising with q We start from a real data point $x_0 \sim q_{\text{data}}(x)$ and define a *forward* (noising) Markov chain: $q(x_{1:T} x_0) = \prod_{t=1}^T q(x_t x_{t-1})$. Each transition is simple (Gaussian), e.g. in DDPM: $q(x_t x_{t-1}) = \mathcal{N}(x_t \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) I)$.
16. Why the “true reverse” is intractable	We start from the forward (noising) Markov chain: $q(x_{1:T} x_0) = \prod_{t=1}^T q(x_t x_{t-1})$; $q(x_t x_{t-1}) = \mathcal{N}(x_t \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) I)$. Marginals of a Markov chain are integrals over the intermediate states In general, the marginal distribution at time t given x_0 is: $q(x_t x_0) = \int q(x_{1:t} x_0) dx_{1:t-1} = \int \left(\prod_{s=1}^t q(x_s x_{s-1}) \right) dx_{1:t-1}$.
17. How p_θ uses a neural network	Since reversed q is intractable, diffusion defines a learned reverse Markov chain: $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} x_t)$; $p(x_T) = \mathcal{N}(0, I)$. What kind of distribution is $p_\theta(x_{t-1} x_t)$?
18. How the neural network is trained in DDPM	This section makes the DDPM training loop explicit: what are the inputs/targets, what network do we use, and what loss do we optimize? We start with a dataset of clean samples: $x_0 \sim q_{\text{data}}(x)$.
19. Representing Time: Sinusoidal time encoding	Diffusion models must be conditioned on the timestep t because denoising at: - small t (slightly noisy) and - large t (almost pure noise) are fundamentally different tasks. Rather than feeding the integer t directly, we map it to a vector embedding $\text{emb}(t) \in \mathbb{R}^d$. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
20. SiLU (Swish) Activation	SiLU stands for Sigmoid Linear Unit. It is also known as Swish (when written as $\text{x} \cdot \text{sigmoid}(\text{x})$). The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

21. 1D DDPM Example Implementation	We intentionally use scalar data x_{inR} (shape $(B,1)$): - Data distribution $q_{data}(x_0)$: a simple two-bump mixture. - Forward diffusion: add Gaussian noise with a schedule β_t . - Model: a tiny MLP that predicts noise $\epsilon_{\theta}(x_t, t)$. - Reverse diffusion (DDPM sampling): start from $x_{TsimN}(0,1)$ and denoise back to x_0 . - Denoising trajectory: show how a single scalar evolves over timesteps. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
22. Forward vs reverse histograms (intermediate timesteps)	These plots make the *Markov chain intuition* concrete: - Forward (noising): start from $x_{0sim} q_{data}$. As t increases, the bimodal density smooths and approaches $N(0,1)$. - Reverse (denoising): start from $x_{Tsim} N(0,1)$. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
23. Single-scalar noising and denoising trajectory	We now track one single scalar sample: - Forward trajectory: start from one clean x_0 and apply the *closed-form forward marginal* $q(x_t x_0)$. - Reverse trajectory: start from pure noise $x_{TsimN}(0,1)$ and apply the DDPM reverse chain. This makes the stochastic Markov chain behavior visually concrete at the *individual sample* level. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
24. What is a U-Net and why is it used in diffusion?	When moving from 1D examples to real images, the neural network used in DDPM is typically a U-Net. A U-Net is a convolutional neural network with: - A downsampling path (encoder) - An upsampling path (decoder) - Skip connections between matching resolutions Its structure looks like a "U": Input Image Conv Downsample Conv Downsample Bottleneck Upsample Time-conditioned denoising networks. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
25. Tiny training example on synthetic 2D blob images	What this example demonstrates: - how to create a simple synthetic blob dataset (32×32 images), - how to train a tiny U-Net to predict noise $\epsilon_{\theta}(x_t, t)$, - how to visualize clean vs noisy vs true noise vs predicted noise. > Important: This cell assumes TinyUNet2D, DiffusionSchedule, and Diffusion2D were defined earlier in the notebook. Visualize the dataset and the reverse process (generation) We now: 1. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
26. DDIM (Denoising Diffusion Implicit Models)	DDIM is best understood as a sampling method for a model trained like DDPM. - Training: unchanged (still predict noise with an MSE loss) $\mathbb{E}[\epsilon_{\theta}(x_t, t)^2]$. - Sampling: choose a different reverse update that can be deterministic (often much faster).
27. Key idea	From the forward marginal $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon$, we can estimate x_0 from (x_t, t) using the network's epsilon prediction: $\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\alpha_t}\epsilon_{\theta}(x_t, t)}{\sqrt{\alpha_t}}$. DDIM proposes a family of reverse updates (indexed by a parameter $\eta \in [0,1]$): $x_{t-1} = \sqrt{\alpha_{t-1}}\hat{x}_0 + \sqrt{1-\alpha_{t-1}-\sigma_t^2}\epsilon_{\theta}(x_t, t) + \sigma_t z$, with $\sigma_t(\eta) = \sqrt{\frac{1-\alpha_{t-1}}{1-\alpha_t}}\eta$. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
28. Animating forward and backwards Diffusion Processes	This notebook heading introduces animating forward and backwards diffusion processes and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 3 nearby code cells for implementation, demonstration, or visualization of this heading.